Web API Hands-On — Complete Answers

Web API using .NET Core with Swagger

Objective

Install Swashbuckle.AspNetCore, configure Swagger in Startup.cs, and verify the Swagger UI on the browser showing controller action methods.

Step 1 – Create / Use an Existing .NET Core Web API Project

```
Terminal / Package Manager Console
dotnet new webapi -n SwaggerDemo
cd SwaggerDemo
```

Step 2 – Install Swashbuckle.AspNetCore NuGet Package

■ NuGet Package: Swashbuckle.AspNetCore Install via Package Manager Console OR .NET CLI

```
Package Manager Console (Visual Studio)
Install-Package Swashbuckle.AspNetCore
```

```
.NET CLI
dotnet add package Swashbuckle.AspNetCore
```

Step 3 – Modify Startup.cs → ConfigureServices Method

Add the **AddSwaggerGen** call inside *ConfigureServices* to register the Swagger generator with title, version, description, contact, and licence details:

```
Startup.cs – ConfigureServices
```

```
using Swashbuckle.AspNetCore.Swagger;

public void ConfigureServices(IServiceCollection
services) { services.AddMvc().SetCompatibilityVersion(
CompatibilityVersion.Version_2_1);

services.AddSwaggerGen(c =>
{
c.SwaggerDoc("v1", new Info
{
Title = "Swagger Demo",
Version = "v1",
Description = "TBD",
TermsOfService = "None",
Contact = new Contact()
{
Name = "John Doe",
Email = "john@xyzmail.com",
Url = "www.example.com"
}, License = new
License()
{
Name = "License Terms",
Url = "www.example.com"
}
});
});
}
```

Step 4 – Modify Startup.cs → Configure Method

Add **UseSwagger()** and **UseSwaggerUI()** middleware to serve the generated JSON and the interactive HTML UI:

Startup.cs – Configure

```
public void Configure(IApplicationBuilder app, IHostingEnvironment env)
{
// ... existing middleware ...

app.UseSwagger();

app.UseSwaggerUI(c =>
{
// Swagger JSON endpoint
c.SwaggerEndpoint("/swagger/v1/swagger.json",
"Swagger Demo");
});

app.UseMvc();
}
```

Using Postman to Test Web API (Employee Controller)

Step 5 – Run the Application and Open Swagger UI

1 **Run the application**
Press F5 (Visual Studio) or run 'dotnet run'. The browser opens at the default URL (e.g. https://localhost:5001/api/values).

2 **Navigate to Swagger UI**
Change URL to: https://localhost:{port}/swagger

3 **Verify page header**
Confirm you see Title='Swagger Demo', Version='v1', Contact info (John Doe / john@xyzmail.com) at the top of the page.

4 **Verify Values controller**
The 'Values' section is auto-listed with all HTTP verb action methods (GET, POST, PUT, DELETE).

5 **Test GET without parameter**
Click the GET row (no parameter) → 'Try it out' → 'Execute'. You should get HTTP 200 with a JSON array ["value1", "value2"].

■ Swagger UI Browser Output

Swagger Demo v1

https://localhost:5001/swagger/v1/swagger.json

Terms of Service | John Doe — www.example.com | License Terms

■ VALUES

GET /api/values	—	200 OK
POST /api/values	—	201 Created
GET /api/values/{id}	—	200 OK
PUT /api/values/{id}	—	204 No Content
DELETE /api/values/{id}	—	204 No Content

Objective

Models/Employee.cs

```
namespace SwaggerDemo.Models
{
public class Employee
{
public int Id { get; set; }
public string Name { get; set; }
public string Department { get; set; }
public double Salary { get; set; }
}
}
```

Controllers/EmployeeController.cs

```
using System.Collections.Generic;
using Microsoft.AspNetCore.Mvc;
using SwaggerDemo.Models;

[Route("api/[controller]")] [ApiController] public class
EmployeeController : ControllerBase { private static List
_employees = new List { new Employee { Id=1, Name="Alice",
Department="IT", Salary=75000 }, new Employee { Id=2, Name="Bob",
Department="HR", Salary=65000 }, new Employee { Id=3,
Name="Charlie", Department="IT", Salary=80000 } };

// GET api/employee
[HttpGet] public
ActionResult> Get() {
return Ok(_employees); }

// GET api/employee/1 [HttpGet("{id}")]
public ActionResult Get(int id) { var emp
= _employees.Find(e => e.Id == id); if
(emp == null) return NotFound(); return
Ok(emp); }
}
```

Postman Tool – Structure Overview

Postman UI Component	What it is / Where to look
Request URL bar	Enter the full URL here (e.g. http://localhost:5001/api/employee)
Method dropdown	Select GET / POST / PUT / DELETE before the URL bar
Headers tab	Add key-value pairs, e.g. Authorization: Bearer <token>
Body tab	Select 'raw' + 'JSON' for POST/PUT; paste JSON payload here
Collections (left)	Save and organise requests; click '+' to add new request to collection
Response Body pane	Bottom pane — shows returned JSON / text from the API
Status code (right)	Green badge (e.g. '200 OK') shown in top-right of response pane

Step-by-Step: Hit GET /api/employee in Postman

1. **Open Postman**
   Launch Postman; click '+ New' → 'Request'.
2. **Select Method**
   From the method dropdown (left of URL bar) select GET.
3. **Enter URL**
   Type: http://localhost:5001/api/employee (use your actual port)
4. **Add Authorization Header (if needed)**
   Click Headers tab → add key 'Authorization', value 'Bearer ' (skip if the endpoint is open)
5. **Click Send**
   Hit the blue 'Send' button. Postman sends the GET request to your running API.
6. **Verify Body**
   The bottom Response pane (Body tab) shows the JSON list of employees.
7. **Verify Status**
   Top-right of the response pane shows '200 OK' in green.

■ Postman Response (simulated screenshot)

```
GET http://localhost:5001/api/employee                    200 OK | 48 ms | 389 B

[
{ "id": 1, "name": "Alice", "department": "IT", "salary": 75000 },
{ "id": 2, "name": "Bob", "department": "HR", "salary": 65000 },
{ "id": 3, "name": "Charlie", "department": "IT", "salary": 80000 }
]
```

Route Attribute, Name Attribute & ActionName

Part A – Modify Route Attribute ('Emp')

Change the controller's [Route] attribute so it answers to **/api/emp** instead of **/api/employee**.

Controllers/EmployeeController.cs — Route changed

```
// BEFORE:
[Route("api/[controller]")] // resolves to /api/employee

// AFTER:
[Route("api/emp")] // resolves to /api/emp
[ApiController]
public class EmployeeController : ControllerBase { ... }
```

■ [controller] token is replaced with the controller class name (minus 'Controller' suffix). Hard-coding the segment ('emp') makes the URL independent of the class name.

Postman after Route change

```
GET http://localhost:5001/api/emp                                    200 OK

Old URL /api/employee → 404 Not Found | New URL /api/emp → 200 OK ✓
```

Part B – Name Attribute on HttpGet (User-Friendly Action Name)

The **Name** property on an Http verb attribute gives the action a user-friendly identifier, enabling link generation via *CreatedAtAction* / *Url.Action* without exposing the C# method name.

Controllers/EmployeeController.cs — Name attribute

```
[HttpGet(Name =
"GetAllEmployees")] public
ActionResult> Get() {
return Ok(_employees); }

[HttpGet("{id}", Name =
"GetEmployeeById")] public
ActionResult Get(int id) { var emp
= _employees.Find(e => e.Id ==
id); if (emp == null) return
NotFound(); return Ok(emp); }

// POST — using CreatedAtAction to return
Location header [HttpPost] public ActionResult
Post([FromBody] Employee emp) { emp.Id =
_employees.Count + 1; _employees.Add(emp); //
Uses the named route to build the Location URL
return CreatedAtAction("GetEmployeeById", new {
id = emp.Id }, emp);
}
```

Part C – ActionName Attribute (Multiple Methods, Same HTTP Verb)

C# does not allow two public methods with the identical signature. **[ActionName]** lets you expose two methods under the same HTTP verb by giving each a distinct action name while the routing differentiates them via template parameters.

Controllers/EmployeeController.cs — ActionName

```
// Method 1: GET /api/emp/active
[HttpGet("active")]
[ActionName("GetActiveEmployees")]
public ActionResult> GetActive()
{
return Ok( employees.Where(e => e.Salary > 70000));
}

// Method 2: GET /api/emp/all
[HttpGet("all")]
[ActionName("GetAllEmployeesList")]
public ActionResult> GetAll()
{
return Ok( employees);
}

// Both are GET but differ in route segment and ActionName,
// so ASP.NET Core does NOT throw AmbiguousActionException.
```

Summary – Route / Name / ActionName

Attribute / Concept	Where used	Purpose
`[Route("api/emp")]`	Controller class	Set URL prefix; override default [controller] token
`[HttpGet(Name="…")]`	Action method	Give action a friendly name for link-generation (CreatedAtA
`[ActionName("…")]`	Action method	Rename action so two methods can share the same HTTP
`ProducesResponseType`	Action method	Document expected HTTP response codes in Swagger

→